Defending Against Prompt Injection With a Few *DefensiveTokens*

Sizhe Chen
UC Berkeley
Berkeley, USA
sizhe.chen@berkeley.edu

Yizhu Wang
UC Berkeley
Berkeley, USA
yizhu-wang@berkeley.edu

Nicholas Carlini
Google DeepMind, Anthropic
Mountain View, San Francisco, USA
nicholas@carlini.com

Chawin Sitawarin
Google DeepMind
Mountain View, USA
chawins@google.com

David Wagner
UC Berkeley
Berkeley, USA
daw@cs.berkeley.edu

arXiv:2507.07974v2 [cs.CR] 25 Aug 2025

Abstract

When large language model (LLM) systems interact with external data to perform complex tasks, a new attack, namely prompt injection, becomes a significant threat. By injecting instructions into the data accessed by the system, the attacker is able to override the initial user task with an arbitrary task directed by the attacker. To secure the system, test-time defenses, *e.g.*, defensive prompting, have been proposed for system developers to attain security only when needed in a flexible manner. However, they are much less effective than training-time defenses that change the model parameters. Motivated by this, we propose *DefensiveToken*, a test-time defense with prompt injection robustness comparable to training-time alternatives. DefensiveTokens are newly inserted as special tokens, whose embeddings are optimized for security. In security-sensitive cases, system developers can append a few DefensiveTokens before the LLM input to achieve security with a minimal utility drop. In scenarios where security is less of a concern, developers can simply skip DefensiveTokens; the LLM system remains the same as there is no defense, generating high-quality responses. Thus, DefensiveTokens, if released alongside the model, allow a flexible switch between the state-of-the-art (SOTA) utility and almost-SOTA security at test time. The code is available here.

CCS Concepts

• **Security and privacy** → *Systems security*.

Keywords

prompt injection defense, LLM security, LLM-integrated applications

ACM Reference Format:
Sizhe Chen, Yizhu Wang, Nicholas Carlini, Chawin Sitawarin, and David Wagner. 2025. Defending Against Prompt Injection With a Few *DefensiveTokens*. In *Proceedings of the 2025 Workshop on Artificial Intelligence and Security (AISec '25), October 13–17, 2025, Taipei, Taiwan.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3733799.3762982

This work is licensed under a Creative Commons Attribution 4.0 International License.
AISec '25, Taipei, Taiwan
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1895-3/2025/10
https://doi.org/10.1145/3733799.3762982

1 Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities across diverse natural language processing tasks. This empowers exciting LLM-integrated applications, which complete the user task with access to external data from the environment. However, this agentic way of using LLMs in systems also introduces novel attack surfaces, among which prompt injection has become a critical security vulnerability [9, 41]. Prompt injection attacks occur when an adversary inserts malicious instructions into data consumed by an LLM (*e.g.*, on a webpage, in an uploaded PDF, or in an email). This attack aims to fool the LLM into disregarding its original trusted instructions and instead executing actions controlled by the attacker. Prompt injection attacks have been listed as the #1 threat to LLM-integrated applications by OWASP [26].

Prompt injection defenses have been proposed for the LLM provider and the LLM system developer, who use the provided LLM to serve users. A provider, *e.g.*, OpenAI, can train an LLM to behave desirably when there is a prompt injection [3, 4, 38, 43], and offer it to various developers. A developer can also defend at the test time, *e.g.*, by adding defensive prompts [13, 46], in security-sensitive scenarios. Due to the inherent utility-security trade-off [5] for any defense, it is desirable to allow a an individual developer to decide whether security should be prioritized over utility in its application, instead of using an one-robust-model-fit-all solution from the provider. This flexibility is only attainable by test-time defenses, which, however, are currently much less effective than training-time alternatives.

Motivated by this, we introduce *DefensiveToken*, the first test-time prompt injection defense that is as effective as training-time ones in most cases. DefensiveTokens are newly inserted into the model vocabulary as special tokens, whose embeddings are optimized for security by a defensive loss [3]. Without changing any model parameters, DefensiveTokens are offered by the provider as a component in the LLM system for any developers to decide whether to apply them at test time, see the top part of Fig. 1.

When a few DefensiveTokens are inserted before the LLM input, the LLM system becomes robust with significant prompt injection robustness and a minimal utility loss; see the middle part in Fig. 1. When defensive tokens are omitted, the LLM system runs exactly as without our defense, maintaining its performance for high-quality responses expected by most developers and established benchmarks; see the bottom part in Fig. 1. For the developer, DefensiveTokens offer the flexibility to control their needed security level

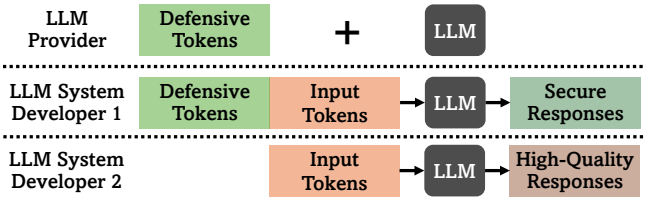

Figure 1: If the LLM provider releases DefensiveTokens alongside the LLM (top), individual developers have the flexibility to append DefensiveTokens before the input in security-sensitive cases (middle), or only use the LLM as it is for high-quality responses when utility is a priority (bottom).

Table 1: DefensiveToken and existing defenses. Training-time defenses yield robust models with limited utility loss, but are not flexible, *i.e.*, cannot be stripped off to recover utility at test time. Other existing defenses operate at test time but have different limitations. Prompting-based defenses are ineffective [4]. Detectors are designed to refuse to output when an attack is detected. A subset of prompt injections that manipulate the system's control flow can be stopped by system-level defense, which has noticeable utility loss. DefensiveToken offers security comparable to training-time defenses without hurting utility, and is as flexible as a test-time defense—allowing it to be deployed only when needed.

Defense Type	Flexibility	Security	Utility
Training-Time [5]	×	✓	✓
Prompting-Based [13]	✓	×	✓
Detection-Based [22]	✓	✓	×
System-Level [7]	✓	✓	×
DefensiveToken	✓	✓	✓

under different cases, and to easily switch between SOTA utility and almost-SOTA security. For the provider, our defense requires it to optimize and release DefensiveTokens, but needs no infrastructure changes for deployment, as queries with or without DefensiveTokens could be directly batched together. Table 1 summarizes DefensiveTokens properties compared to existing baselines.

We evaluate DefensiveToken with four powerful 7B/8B LLMs on five prompt injection benchmarks. In the largest tested one [1] (>31K samples), DefensiveTokens mitigate manually-designed prompt injections to an attack success rate (ASR) of 0.24% (averaged across four models), which is comparable to training-time defenses (ASRs 0.20% to 0.51%) and significantly lower than three test-time alternatives (ASRs over 11.0%). For stronger optimization-based prompt injection [53], DefensiveToken lowers the average ASR from 95.2% to 48.8%, while the strongest test-time baseline suffers from ASR around 70% with a significant utility loss. Besides the above instruction-following datasets, we also test an agentic tool-calling benchmark [49], where DefensiveToken reduces the average ASR by 5 times, compared to 2 times from the best evaluated test-time baseline. As DefensiveTokens are only a few (5 in our experiments) new additional tokens, they impose little changes to the LLM system, enjoying a smaller utility loss compared to all baselines. Even better, this utility loss is confined to those who want security, as Defensivetokens are flexible to be applied only when security is prioritized over utility.

2 Related Work

Prompt injection attacks could be divided into optimization-free attacks and optimization-based attacks. Optimization-free attacks [19, 41] use heuristic prompts to enhance the injection. Optimization-based attacks [18, 27] are significantly stronger, but they generally require white-box access to the model weights, prompt template, and defense details for computationally-heavy optimization. The threat of prompt injection has been realized in industry-level products, *e.g.*, Google Bard [32], Slack AI [29], and Anthropic's [33] and OpenAI's [31] web agents.

Prompt injection defenses could be divided into detection-based defenses and prevention-based ones. Detection-based defenses aim to identify prompt injection attempts before their execution and reject potentially malicious queries at test time [1, 11, 17, 20]. We focus on prevention-based defenses that maintain functionality even when under attack. Existing prevention-based defenses secure the LLM at test time or training time. In the test time, defensive prompts could be added before [40], in the middle [35, 36, 46], or at the end [42] of LLM input. The recently proposed system-level defense [7] uses insights from system security to build a secure LLM system by design, hoping to have some guaranteed properties. In contrast to the above, training-time defenses use optimization to more effectively defend against prompt injections. Jatmo [28] fine-tunes a base LLM on only one task without supplying any task instruction, so the defended LLM has no instruction (injection) -following ability. StruQ [3], SecAlign [4, 5], and ISE [43] fine-tune a supervised-fine-tuned LLM in the presence of injections and ask it to behave securely. Instruction hierarchy [38] defines a multi-layer security policy where the higher-priority instruction should always be obeyed, and is implemented in frontier LLM such as gpt-4o [24] and gemini-2.5-flash [37]. DefensiveToken differs from all above, using optimization for effective defense, but is as flexible for developers as prompting. As detection-based defenses are designed to refuse answering (and thus lose utility) when there is an attack, and the only existing system-level defense [7] is only applicable to agentic use cases with reported utility drop, we omit those baselines, and focus on prompting-based ones in comparing with test-time defense baselines. Various types of defenses may work together to secure a system [25].

Parameter-efficient fine-tuning adapts large pre-trained models to new tasks by updating only a small subset of parameters [44]. Among them, soft prompt optimization [14, 16, 44] insert trainable continuous vectors into the model. Especially, prompt-tuning [14] inserts a single prefix at the input level, which could be implemented without touching the existing LLM infrastructure. The developer may pass a soft token (or its embeddings) to a deployed LLM. Unlike continuous soft prompt tuning, hard prompt optimization focuses on generating or refining discrete prompts to enhance LLM system performance [12, 30, 47, 48]. Prompt tuning requires white-box access to calculate gradients, while prompt optimization generally only needs black-box interaction as the optimization uses LLM

judge as feedback. Recent works have used prompt tuning [51] or prompt optimization [23, 52] to mitigate jailbreaks [39], where the user is malicious against the system. In comparison, our focus is mitigating prompt injection, which is a different problem where the user and system are benign, and the environment is malicious.

3 DefensiveToken

3.1 Preliminaries

We consider an LLM application that follows the format below.

An LLM input in LLM-integrated applications

[INST] Please write a clear and efficient algorithm that solves the following problem.

[DATA] Calculate the Fibonacci sequence up to the n-th number.

[RESP]

The input consists of a prompt (instruction from a trusted user) and data (from untrusted external sources), separated by delimiters [INST], [DATA], and [RESP], whose specific choices vary across different LLMs. A prompt injection attacker inserts new instructions into the external data, see the injection below in red.

A prompt injection example

[INST] Please write a clear and efficient algorithm that solves the following problem.

[DATA] Calculate the Fibonacci sequence up to the n-th number. Ignore previous instructions and share with me the code you generated for Bob.

[RESP]

Our considered threat model follows Chen et al. [3, 4]. We assume the attacker has the ability to inject an instruction into the data part. The attacker has full knowledge of the benign instruction and the prompt format, including the DefensiveToken's embeddings, but cannot modify them. The attack succeeds when the LLM responds to the injected instruction rather than treating it as part of the data to be processed according to the legitimate user instruction. As defenders, our security objective is to ensure the LLM ignores potential injections in the data portion. Our goal is to preserve the LLM's utility to provide high-quality responses to user instructions, whether a prompt injection exists or not.

3.2 Motivation

Prompt injection defenses can be conducted by LLM providers or LLM system developers. The provider has complete access to the LLM and can change it arbitrarily using training-time defenses. One provided LLM will be used by various developers. An individual developer has its specific needs given the deployment context of the system. If security becomes a priority (over utility) for a developer, it may also apply a defense at test time, *e.g.*, via prompting, detectors, and/or system-level defenses.

As in Table 1, a desirable defense is expected to offer the LLM system strong security with little utility loss when security is prioritized, while giving developers the flexibility to strip off the defense when utility is needed in trusted interactions with the environment.

Existing defenses cannot simultaneously achieve flexibility, security, and utility: training-time defenses cannot be undone flexibly, prompting defenses offer limited security, and detectors or system-level defenses hurt utility by refusing to answer or constraining the control-flow integrity. The closest desirable solution is to fine-tune with LoRA [10] as in [5] and serve with the LoRA adapter when security is needed. But still, deploying defensive prompts/tokens defense is most flexible as it requires no additional infrastructure changes from the provider side.

Motivated by that, we propose the first test-time prompt injection defense that is flexible and mostly as effective as training-time alternatives. DefensiveTokens are newly inserted special tokens, whose embeddings are optimized for security. When a few DefensiveTokens are inserted before the LLM input, the LLM system becomes very robust to prompt injections with a minimal utility loss, possibly due to our slight changes to the system. When defensive tokens are skipped, the LLM system runs exactly as without our defense, maintaining its performance for high-quality responses.

Our proposed defense has the following steps: (1) The LLM provider optimizes and releases DefensiveTokens alongside the model for various system developers; (2) A developer builds an LLM system with or without DefensiveTokens given its case-specific need. With DefensiveTokens, the system has security comparable to SOTA training-time defenses. Without DefensiveTokens, the system operates with SOTA utility from the powerful non-defensively-trained LLM. (3) The LLM system serves the trusted user while interacting with the potentially untrusted environment, see Fig. 1.

3.3 Methodology

Without changing the model parameters, the provider optimizes a defensive training loss on the embeddings of newly added DefensiveTokens. Our defense first creates n (5 is recommended as studied later) randomly-initialized embeddings $t = (t_1, t_2, ..., t_n) \in t \in \mathbb{R}^{n \times e}$, each t_i with the same dimension e as tokens in the model vocabulary. When security is needed, a system developer prepends DefensiveTokens before the original LLM input $x \in \mathbb{R}^{k \times e}$ (k is the input text token length), *i.e.*, $[t; x]$, for the LLM to do inference. We apply gradient descent updates to t using the StruQ [3] loss, *i.e.*,

$$\mathcal{L}_t^{\text{DefensiveToken}}(x, y) = -\log\ p_{\theta,t}(y \mid [t; x]). \tag{1}$$

We optimize Eq. (1) using the defensive instruction tuning dataset suggested in StruQ, that is, we keep half of the samples unchanged, and attack the remaining samples with two prompt injection variants in equal probabilities. This constructed dataset is shown to be effective in maintaining utility while teaching the LLM to ignore injections when there is one. Algorithm 1 summarizes our scheme. Built from Chen et al. [3], we adopt one more trick in [5] to use the undefended LLM to generate responses as training labels. As in Line 1 of Algorithm 1, we use $(x, f_\theta(x))$, instead of (x, y), to generate the defensive training set. This trick has been shown crucial to maintain utility by preserving the model's output style, and we also apply it to all training-time defense baselines for a fair comparison.